Decidability Before Maintenance: What Actually Determines Whether an LLM Agent Uses the Right Memory

Han Li

Chengdu Technological University, Chengdu, China · 2499371088@qq.com

Abstract

LLM agents increasingly keep long-term memory across sessions, yet it is unclear what determines whether a memory is used correctly when records conflict. We introduce a controlled task in which a store holds both an outdated and an updated value for the same slot, and the agent must report the current value. Across 37 conditions and roughly 51,000 queries on four model configurations, maintenance policies (recency, importance, decay-based review) explain far less variance than three presentation-level factors that we group under decidability. With reliable timestamps, staleness is essentially eliminated (at most 0.2%). Without timestamps, position alone flips the error rate between 0% and 100%, and the direction is model-specific: the DeepSeek models select the last record, GLM-4-Flash the first, and GLM-4-Plus follows no stable rule. Wording consistency further modulates the effect, and the position of the value inside a sentence matters more than the source label or the sentence form. Review policies help exactly as much as they remove conflicting records: the stale rate equals one minus archive coverage. A real-log replication reproduces the pattern, and a decidability-aware memory layer built from these rules reaches 100% exact answers on an end-to-end multi-turn task where an append-only log reaches 0%. We argue that memory systems should treat decidability as a first-class property, and that memory benchmarks must report timestamps, ordering and wording uniformity.

Keywords: LLM agents; long-term memory; memory maintenance; position bias; evaluation

1 Introduction

An agent that remembers is an agent that keeps a store of facts across sessions [9, 10]: the user's port number, the project directory, the model the workflow is pinned to. Such stores are not append-only. Facts get superseded, and both the old and the new record usually remain, because deleting is lossy and cheap storage makes retention the default. When the agent later answers a question, it must decide which record is current.

A natural assumption is that better memory maintenance fixes this: rank records by recency, by importance, or by a forgetting curve, and surface the right one. A large body of work follows this assumption, from decay-weighted retrieval [1, 2] to paged memory hierarchies [3] and production memory layers [8]. We test that assumption here, and it accounts for remarkably little.

We construct a controlled task in which a memory store holds a chain of two records for the same slot, an outdated value and an updated one, and the model is asked for the current value. The task is deliberately minimal: single-hop, short context, one question. If maintenance policy mattered, this is where it should show. Across 37 conditions and roughly 51,000 queries on four model configurations, we find the opposite: what decides the answer is not how memory is maintained but whether the two records are presentationally decidable.

Three presentation-level factors turn out to dominate. Reliable timestamps all but eliminate staleness (at most 0.2%). Take the timestamps away, and the order of records alone flips the error rate between 0% and 100%; which way it flips depends on the model, with the DeepSeek models selecting the last record, GLM-4-Flash the first, and GLM-4-Plus neither. Wording consistency then decides how strictly that positional rule is applied, and the position of the value inside a sentence matters more than the source label or the sentence form. Maintenance policies, by contrast, help exactly as much as they remove conflicting records, and no more: the observed error rate equals one minus the fraction of conflicts archived, across all four policies.

These results reframe a practical question. Instead of asking which rehearsal policy is best, memory systems should first ask whether a record can be adjudicated at all, and design the store so that it can. We close with a short design recipe and with the measurement practices that would have caught these effects earlier.

Contributions. We build a controlled task and a 37-condition design that separates maintenance policy from presentation. We show that timestamps dominate, and that without them position determines the answer, in a direction that depends on the model. We decompose wording effects and find that the position of the value inside a sentence matters more than its source label or its form. We show that the benefit of maintenance equals its conflict-removal rate, and replicate the pattern on a real-log corpus. We also propose a decidability-aware memory layer, and check it end-to-end.

2 Related Work

Memory for LLM agents. MemGPT [3] treats the context window as main memory and pages information to and from external storage. Generative Agents [1] score memories by recency, importance and relevance, where recency is an exponential decay. MemoryBank [2] models memory strength with an Ebbinghaus-style forgetting curve. Production memory layers [8] add extraction, consolidation and graph structure. These systems differ in storage and retrieval, but they share an assumption we examine here: that better ranking or better rehearsal yields better answers when records conflict.

Long-term memory evaluation. LoCoMo [5] and LongMemEval [6] probe recall and reasoning over long conversational histories. They measure whether an agent can find and use information, usually with a single ground-truth record per fact. Our task differs in the one place that matters for staleness: the store deliberately contains a contradicting record, and the question is which one the model believes.

Position effects. That language models are sensitive to where relevant text appears is well established, including the well-known degradation for material in the middle of long contexts [4]. Our results extend this to conflicting memory: position is not merely a retrieval prior that shifts accuracy by a few points, but in the absence of timestamps it can fully determine the answer, and its direction differs across models.

Maintenance and spaced repetition. Spaced-repetition schedulers such as SM-2, FSRS and half-life regression [7] decide when a learner should review an item. We borrow the framing but apply it to an agent, and we report a negative result for the scheduling part: the benefit of review is attributable to the removal of conflicting records, not to rehearsal or to the decay signal that selects items.

3 Task and Experimental Design

3.1 Task

Each memory store contains N fact chains. A chain consists of an outdated record and an updated record for the same (entity, attribute) slot, written at day t_old and t_new with t_old < t_new, plus distractor records that consume budget. The query time is fixed at day 200; the update is placed d days before the query, with d in {1, 7, 30, 90}. Distractor entities are never queried.

For each chain the model receives the records retrieved for that entity, rendered either as a short template ("[2026-05-28] service-00·port = 8000") or as a natural sentence, and answers one question: what is the current value? An answer equal to the updated value is correct; an answer equal to the outdated value is a stale answer; anything else is counted as unknown. The primary metric is the stale rate.

3.2 Maintenance policies

We compare four policies under an identical review budget of 40% of the store's characters: (i) no review; (ii) recency-first; (iii) importance-first; and (iv) decay-first, which selects the records with the lowest retention $w = \exp(-\Delta t / H)$, with half-life $H \in \{30, 12, 6\}$ days by importance band and a +0.35 rebound when a record is mentioned again. Reviewing a record means checking it against newer records: if it has been superseded it is archived, otherwise its timestamp is refreshed. Retrieval after review follows a fixed rule, so the only thing that varies across policies is which records were reviewed.

3.3 Factors

Time representation: absolute dates, a relative notation ("day N"), or no timestamp at all. Context order: newest first, oldest first, or shuffled. Wording consistency: an identical template for all records, natural sentences with a uniform form, or natural sentences with mixed forms; three single-factor variants change only the source label, only the sentence form, or only the position of the value within the sentence. Models: deepseek-chat, deepseek-reasoner, glm-4-flash and glm-4-plus, all at temperature 0 with short outputs (800 tokens for the reasoning model).

3.4 Real-log corpus

To check that the results are not an artifact of synthetic strings, we mine ten days of the first author's agent session logs for a configuration value that genuinely changed over time: the local port of a desktop agent UI, observed 25 times and taking 14 distinct values. Consecutive observations form 13 update chains. Memory records are the original log lines, verbatim; only the question and the scoring are ours.

3.5 Statistics

Each condition is run with 40 chains, 4 delays and 3 random seeds (13 chains and 3 seeds for the real-log corpus). The whole grid fits on one workstation and costs a few dollars in API calls. We report means over delays with 95% confidence intervals computed across seeds using the t-distribution. Because the task is deterministic and the contrasts are large, we report effect sizes directly rather than relying on significance tests.

4 Results

4.1 Timestamps remove the problem

Table 1 reports the main grid on the template corpus. With absolute dates, the model is essentially never wrong, and no maintenance policy improves on that: the stale rate is 0.2% for no review and 0.0% for all three policies. The ceiling is not an artifact of easy questions; it is what dates are for. A relative notation is already much weaker: it averages 15.8% without review and rises to 50.8% at a 90-day delay, which suggests that models do not reliably map "day 110" and "day 65" onto a chronology.

Table 1. Stale rate (%) on the template corpus, deepseek-chat, 40 chains × 4 delays × 3 seeds. Lower is better.

Condition	No review	Recency	Importance	Decay
Dates, newest first	0.2	0.0	0.0	0.0
Dates, oldest first	0.0	0.0	0.0	0.0
Relative ("day N")	15.8	4.8	4.2	4.8
No timestamp, newest first	100.0	100.0	55.4	47.5
No timestamp, oldest first	0.0	0.0	0.0	0.0

4.2 Without timestamps, position decides, and the direction is model-specific

Removing timestamps but keeping identical wording produces a stark result. With the updated record first, the stale rate is 100%; with the updated record last, it is 0%. All 480 queries in each no-review condition agree with the position rule, and a shuffled order falls between the extremes (34.8%, measured under mixed wording), consistent with order being the operative cue. The model is not reasoning about which record is newer; it is reading a position.

Table 2 shows that the direction of this rule is a property of the model, not of the task. Both DeepSeek models pick the last record. GLM-4-Flash picks the first: it is 0% stale when the update is first and 100% when it is last. GLM-4-Plus does something else again, landing at 49.4% and 30.4%, i.e. above chance but with no rule that we can identify. Adding dates returns all four configurations to 0%, which isolates the failure to the absence of a decidable cue.

Table 2. Stale rate (%) without timestamps, by model and record order, with the no-review baseline. The real-log row uses 13 chains of genuine configuration changes.

Model / corpus	Update first	Update last	With dates
deepseek-chat	100.0	0.0	0.2
deepseek-reasoner	100.0	—	0.0
glm-4-flash	0.0	100.0	0.0
glm-4-plus	49.4	30.4	0.0
deepseek-chat, real logs	89.1	8.3	0.0

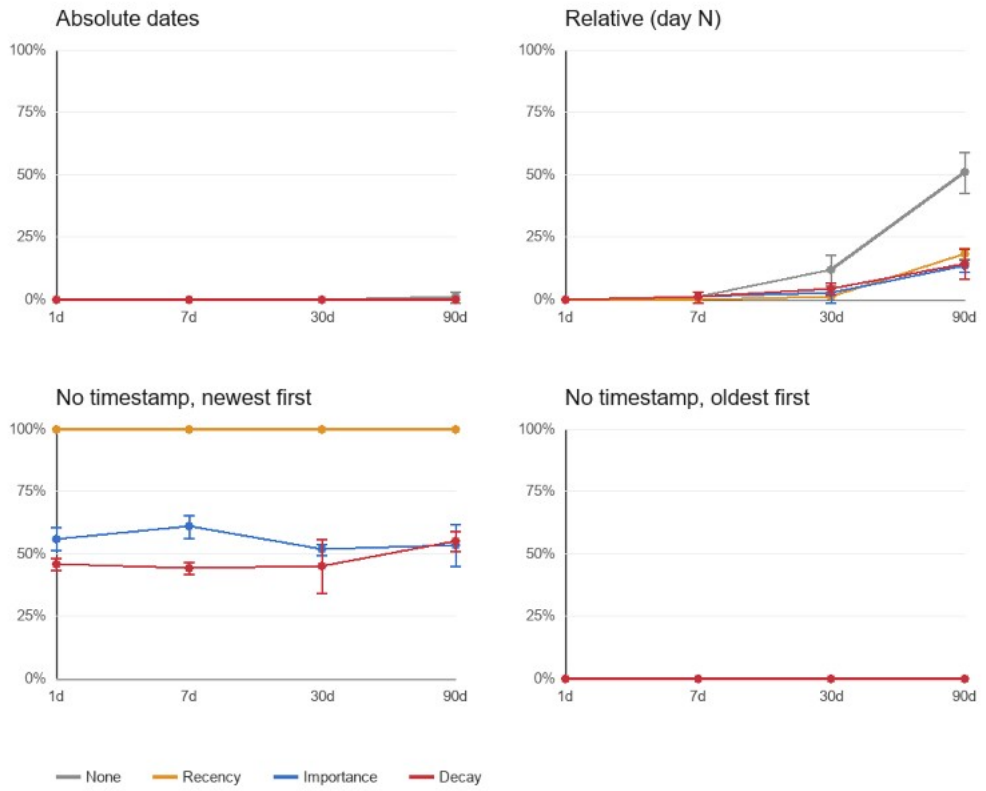


Figure 1. Stale rate by delay under four presentation conditions (deepseek-chat, 40 chains × 3 seeds). Error bars are 95% confidence intervals across seeds.

4.3 Wording matters, and value position matters most

Identical wording is what makes the position rule deterministic, so we ask which aspect of wording breaks it. Starting from the uniform baseline (100% stale for deepseek-chat, 0% for glm-4-flash), we vary one dimension at a time (Table 3). For deepseek-chat, changing only the source label lowers staleness to 90.4%, changing only the sentence form to 85.6%, and changing only the position of the value inside the sentence to 70.2%; mixing all three reaches 65.0%. The effects are roughly additive (9.6 + 14.4 + 29.8 ≈ 53 points), with some overlap when combined.

The same manipulation on glm-4-flash is even more diagnostic. Source labels and sentence forms leave its first-record rule untouched (0.0% in both), but moving the value within the sentence pushes staleness to 35.0%. Value position is therefore the one wording lever that acts on both models, which is why we treat it as the primary surface factor rather than a stylistic detail.

Table 3. Stale rate (%) under single-factor wording perturbations, no timestamps, update first, no-review baseline.

Wording variant	deepseek-chat	glm-4-flash
Uniform (baseline)	100.0	0.0
Only source label varies	90.4	0.0
Only sentence form varies	85.6	0.0
Only value position varies	70.2	35.0
All three mixed	65.0	—

4.4 Maintenance is conflict removal, nothing more

Why do review policies help at all in Table 1? The mechanism is visible in the archived counts. In the no-timestamp, update-first condition, all four policies review essentially the same number of records within the same budget (0, 41.3, 41.0 and 41.5), but they archive very different numbers: 0, 0, 17.8 and 21.0 records, i.e. 0%, 0%, 44.5% and 52.5% of the 40 conflicting pairs. The stale rates are 100.0%, 100.0%, 55.4% and 47.5%.

The relationship is exact: stale rate = 1 − archive coverage. Once a conflict has been removed, the context contains a single record and the answer is necessarily correct; while it remains, the position rule decides and the answer is necessarily the outdated one. The decay signal is therefore not doing any work of its own: importance-first, which ignores decay entirely, lands on the same line. Maintenance helps only to the extent that it deletes contradictions.

Table 4. Review actions and outcomes under an equal review budget, template corpus without timestamps, deepseek-chat. Coverage is archived conflicting pairs divided by 40.

Policy	Reviewed	Archived	Coverage	Stale rate
No review	0.0	0.0	0%	100.0
Recency first	41.3	0.0	0%	100.0
Importance first	41.0	17.8	44.5%	55.4
Decay first	41.5	21.0	52.5%	47.5

4.5 Real logs replicate the pattern

The last row of Table 2 replaces synthetic values with thirteen genuine update chains extracted from session logs, with records rendered as the original log lines. The pattern survives: 89.1% stale when the updated record comes first, 8.3% when it comes last, and 0.0% with dates. The rates are slightly lower than the synthetic ones, plausibly because the real lines differ slightly in form, but the intervention is the same and so is the ordering of conditions.

4.6 The cue sits at the end of the context

The position result raises a mechanistic question: is the model sensitive to the order of the two records, or to how close the last value is to the question? We separate the two by inserting k neutral sentences that contain no digits ("routine inspection note; machine room temperature normal"). In the gap condition the fillers sit between the two records; in the tail condition they follow both. If the cue were the separation between the records, the tail condition should be inert; if it were proximity to the question, the gap condition should be inert.

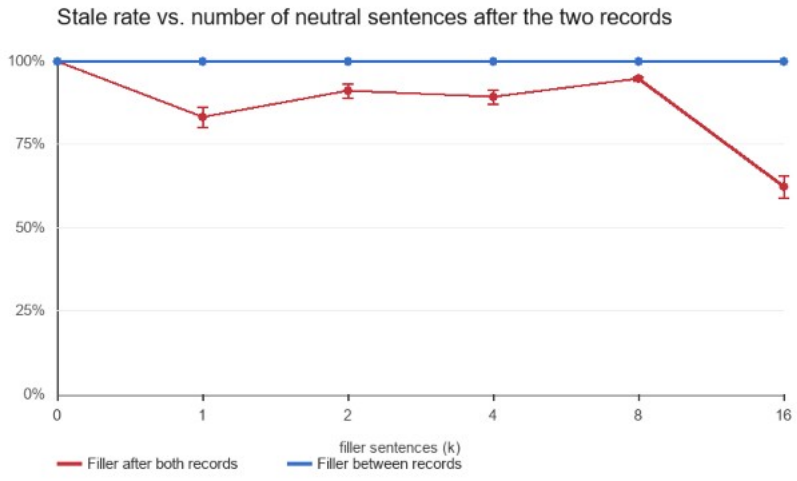


Figure 2. Stale rate as neutral filler sentences are added after both records (red) or between them (blue); no timestamps, uniform wording, deepseek-chat, 40 chains × 3 seeds, no review. Error bars are 95% CIs across seeds and delays.

The gap condition is exactly inert: staleness stays at 100% for every k we test (1 and 8), because the outdated value is still the last fact in the context. The tail condition breaks the rule immediately: staleness falls to 83.1 ± 3.0 at k = 1, stays between 89.4 and 94.8 for k ≤ 8, and drops to 62.3 ± 3.2 at k = 16 (Figure 2). The relation is not a smooth decay: appending text first weakens the cue slightly and then, past a threshold, degrades it substantially.

We read this as evidence that the cue is anchored at the end of the context: what matters is which value sits nearest the question, and that anchor is fragile to any text appended after it. It also rules out the alternative that the model compares the two records and prefers the later one by some distance metric, since separating the records changes nothing at all.

5 A Decidability-Aware Memory Layer

The measurements above suggest a small set of rules for memory systems. We state them as a layer that sits between the store and the prompt, and we report an end-to-end check of a prototype.

5.1 Principles

P1, time-stamp everything. Every record carries the time at which it became true, and the timestamp is rendered with the record. A record without a time is not decidable, and a date is worth more than any ranking signal.

P2, archive conflicts, do not rehearse them. When a new record supersedes an old one, mark the old record as archived rather than keeping it live and reviewing it. Section 4.4 shows that this is the only mechanism through which review policies help, and doing it eagerly is cheaper than discovering the conflict later.

P3, place the authoritative record at the model-specific end. When timestamps are absent or untrusted, the model falls back to a positional rule whose direction differs across models; the placement should therefore be measured per model rather than assumed. Uniform templates keep that rule deterministic, which is desirable only if the placement is correct.

5.2 Interface

The layer exposes three operations. `write(slot, value, t)` appends a record and marks any live record for the same slot as superseded. `render(budget, model)` emits the live records in the model's preferred order with uniform templates and timestamp prefixes, truncating by relevance when the budget is exceeded. `maintain(budget)` spends the maintenance budget on archiving, never on rehearsal, and returns the number of conflicts removed so that operators can see the resulting coverage.

5.3 End-to-end check

We built a prototype of roughly two hundred lines and a multi-turn configuration task: five services, two attributes each, ten updates spread over eight turns, followed by a request for the current deployment manifest. The four designs share the same record set and differ only in rendering and conflict handling; each is evaluated on 40 task instances (20 scenarios × 2 seeds), scoring exact manifest match, field accuracy, and the fraction of fields that carry a superseded value.

Table 5. End-to-end multi-turn configuration task: exact manifest match, field accuracy, and stale fields, 40 instances per design.

Design	Exact	Field acc.	Stale fields
Append-only, newest first	0.0%	44.5%	55.5%
Append-only, oldest first	100.0%	100.0%	0.0%
Timestamped, all records	100.0%	100.0%	0.0%
DAML (archive conflicts)	100.0%	100.0%	0.0%

The naive design is decided by order, not content: the same records give 0% or 100% exact answers depending on which end the newest record occupies. Adding timestamps removes the dependence entirely. DAML matches the timestamped design on accuracy while emitting 10 of 18 records, a 44% reduction in memory tokens, because superseded records are archived rather than carried.

The pilot is small and the task is synthetic; we present it as an existence proof that the rules survive an end-to-end setting, not as a benchmark. A full evaluation with multi-hop tasks, more models and a real deployment is left to future work.

6 Discussion

Decidability is a design property, not a model capability. A memory record is decidable for a given presentation if the context contains enough information to determine which of two contradicting records is current. Dates make a store decidable. Position does not: it converts an epistemic question into a surface heuristic, and the heuristic points in different directions for different models. Wording uniformity is what makes the heuristic deterministic, which is why the same store is 100% stale for one model and 0% for another.

The practical consequences are concrete. Store a timestamp with every record and render it; a fact without a date is not one an agent can use reliably. Pick the placement of the newest record by measurement rather than intuition, since putting the latest record last is right for the DeepSeek models we test and exactly wrong for GLM-4-Flash. Keep record templates homogeneous if you rely on order, and remember that the position of the value inside the sentence is itself a lever. And budget maintenance for conflict resolution rather than rehearsal: a review policy that never archives superseded records will not improve accuracy, however principled its ranking signal.

The results also suggest a measurement practice. Evaluations of agent memory usually hold presentation constant and vary the policy; our data show that presentation can account for the full range of outcomes. Benchmarks that inject contradictions should therefore report, at minimum, whether timestamps are present, how conflicting records are ordered, and whether their wording is uniform. Without those three numbers, a memory-system comparison cannot be interpreted.

One further result deserves emphasis: the negative result for decay. Decay-first review performs well in our grid, but only because it happens to select old records, which are the ones that carry conflicts. Any policy with the same bias would do as well. Claims that forgetting curves improve agent memory [2] should therefore be tested against a conflict-removal baseline before being attributed to the curve.

7 Limitations

Our task is deliberately minimal: single-hop facts with one contradicting record, Chinese prompts, and short answers. Multi-hop questions, longer conflict chains, and tasks that require aggregating several records may behave differently, and we make no claim about them. The real-log corpus is small (13 chains) and covers a single attribute (a local port). Four model configurations from two providers cannot establish how widespread each positional rule is; the finding that the direction differs across models is robust within our sample, but the population of models is not. Review is implemented as a deterministic reconciliation rule, which isolates the selection question but abstracts away the cost and failure modes of an LLM actually rewriting memory. The near-50% behavior of GLM-4-Plus remains unexplained, and we report it as observed rather than interpret it.

8 Conclusion

When an agent's memory contains contradicting records, the answer it gives is decided less by how the memory is maintained than by whether the records can be adjudicated at all. Dates decide it; order decides it in a direction that depends on the model; wording uniformity decides whether the order rule is applied deterministically. Maintenance policies matter only as conflict removal, and their apparent benefits vanish once that is controlled. Memory systems should be built, and evaluated, with decidability as the first-class property.

References

Park J. S., O'Brien J. C., Cai C. J., et al. Generative Agents: Interactive Simulacra of Human Behavior. UIST 2023. https://arxiv.org/abs/2304.03442

Zhong W., Guo L., Gao Q., et al. MemoryBank: Enhancing Large Language Models with Long-Term Memory. AAAI 2024. https://arxiv.org/abs/2305.10250

Packer C., Wooders S., Lin K., et al. MemGPT: Towards LLMs as Operating Systems. 2023. https://arxiv.org/abs/2310.08560

Liu N. F., Lin K., Hewitt J., et al. Lost in the Middle: How Language Models Use Long Contexts. TACL 2024. https://arxiv.org/abs/2307.03172

Maharana A., Lee D. H., Tulyakov S., et al. Evaluating Very Long-Term Conversational Memory of LLM Agents. 2024. https://arxiv.org/abs/2402.17753

Wu D., Wang H., Yu W., et al. LongMemEval: Benchmarking Chat Assistants on Long-Term Interactive Memory. 2024. https://arxiv.org/abs/2410.10813

Settles B., Meeder B. A Trainable Spaced Repetition Model for Language Learners. ACL 2016. https://aclanthology.org/P16-1177/

Lewis P., Perez E., Piktus A., et al. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. NeurIPS 2020. https://arxiv.org/abs/2005.11401

Shinn N., Cassano F., Gopinath A., et al. Reflexion: Language Agents with Verbal Reinforcement Learning. NeurIPS 2023. https://arxiv.org/abs/2303.11366

Wang G., Xie Y., Jiang Y., et al. Voyager: An Open-Ended Embodied Agent with Large Language Models. 2023. https://arxiv.org/abs/2305.16291

Borgeaud S., Mensch A., Hoffmann J., et al. Improving Language Models by Retrieving from Trillions of Tokens. ICML 2022. https://arxiv.org/abs/2112.04426

Ebbinghaus H. Über das Gedächtnis. 1885. https://archive.org/details/berdasgedchtni00ebbi